Provable Security:

What do we know and How can we be sure?

William Bardon
UMBC, CSEE Department
1 July 2026
AvengerCon X 2026

Introduction

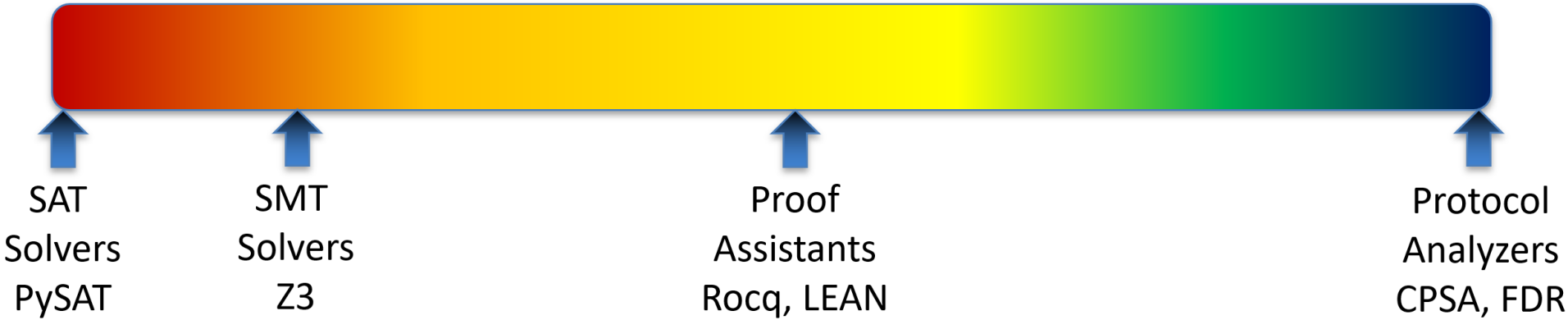
- AI code generation
- Coders become Code Reviewers
- Scale problem

Proof for Coders

- Clearly state security goals
- Consider how actions affect those goals
- Question assumptions
- Tools can examine all possible sequences of events

Spectrum of FM Tools

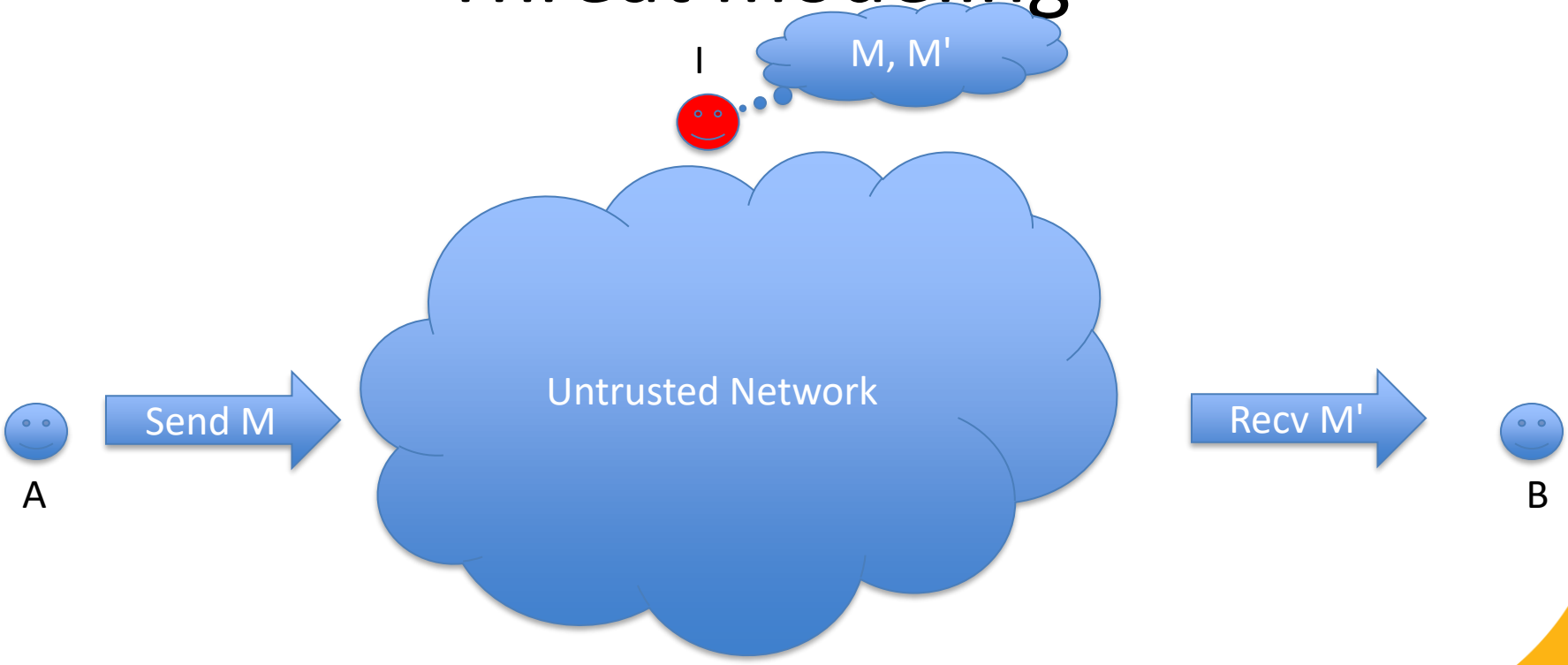
- Satisfiability (SAT) Solvers and Satisfiability Modulo Theories (SMT)
  - CaDiCaL (Conflict-Driven Clause Learning), PySAT, Z3, cvc5
- Theorem Provers/Proof Assistants
  - LEAN, Rocq
- Specialized Tools
  - CPSA (Cryptographic Protocols), FDR (Communicating Processes)



Security Goals

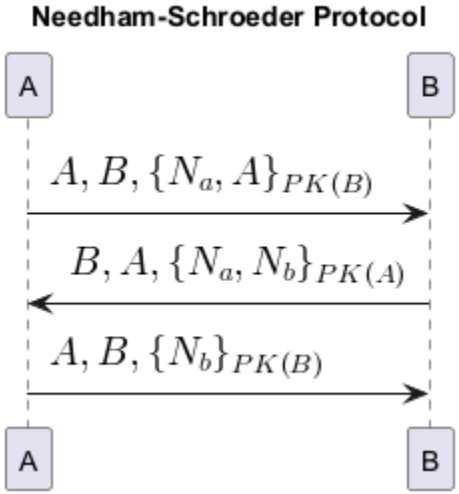
- Goals of Confidentiality, Integrity, Authentication
- Precondition assumes goal is met
- Action transforms system state
  - intentional results – share info with ally
  - unintentional results – reveal info to enemy
- Post-condition confirms security goal still met

Threat modeling

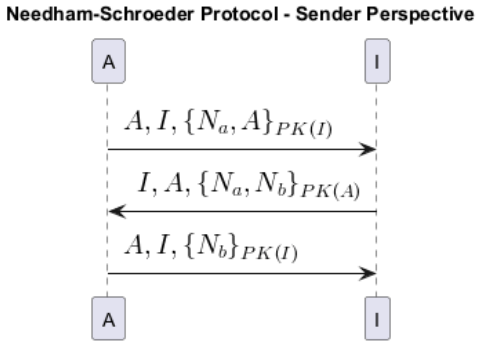


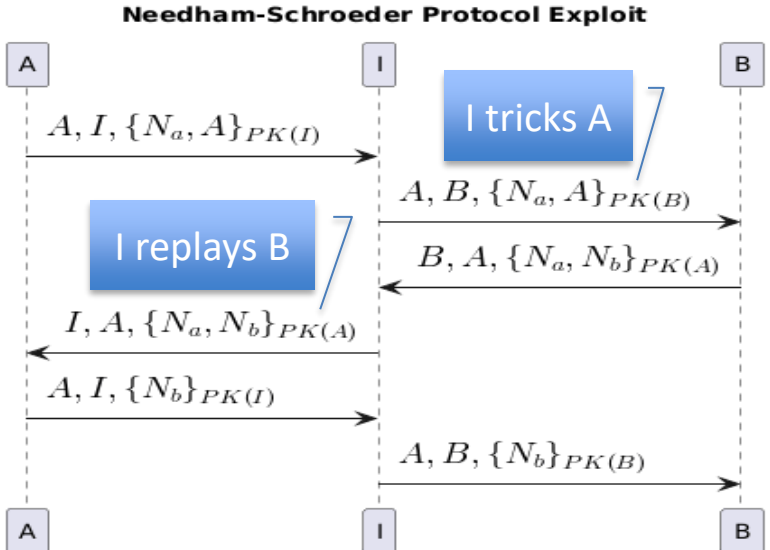
Dolev-Yao threat model

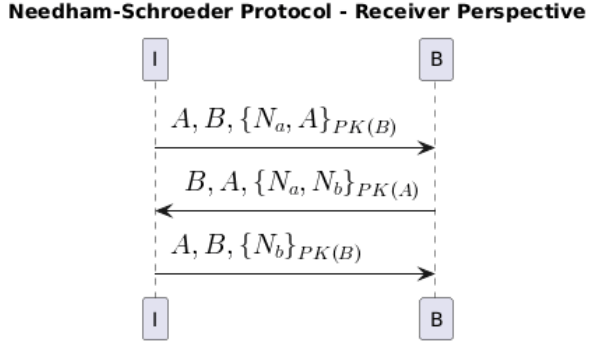
Needham-Schroeder



Lowe's Exploit







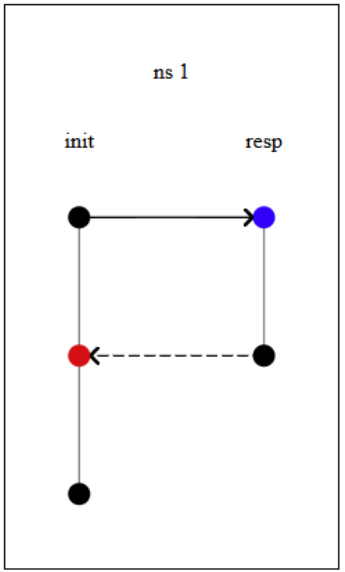
Mitre's CPSA

```
(herald "Needham-Schroeder Demo June 2025"
    (comment "Example of analyzing NS.")
)

(defprotocol ns basic ;; Use basic cryptoalgebra. Other option is Diffie-Hellman (DH)
         (defrole initiator
        (vars (nonce_a nonce_b data) (name_a name_b name))
        (trace
            (send (enc nonce_a name_a (pubk name_b)))
            (recv (enc nonce_a nonce_b (pubk name_a)))
            (send (enc nonce_b (pubk name_b)))
        ))
         (defrole responder
        (vars (nonce_a nonce_b data) (name_a name_b name))
        (trace
            (recv (enc nonce_a name_a (pubk name_b)))
            (send (enc nonce_a nonce_b (pubk name_a)))
            (recv (enc nonce_b (pubk name_b)))
        ))
)

;; Initiator "perspective".
(defskeleton ns
    (vars (name_a name_b name) (nonce_a data))
    (defstrandmax initiator (name_a name_a) (name_b name_b) (nonce_a nonce_a))
    (non-orig (privk name_a) (privk name_b))
    (uniq-orig nonce_a)
)
```

Item 1, Parent: 0, Child: 2.



CPSA = Cryptographic Protocol Shapes Analyzer

Limits of Provers

- Some statements are undecidable
- Non-polynomial resource requirements
  - May run out of memory and crash
  - May run for a very long time
- Insufficient constraints

Hybrid Approach

- AI translates code or spec to model
- Prover verifies model
  - Rejection case: AI revises model
  - Insufficient memory or timeout: AI simplifies model
  - Stop when? How good is good enough?
- AI translates model back to spec or code

Summary

- AI Code is here to stay
- Fixing AI generated code is harder than writing it by hand
- Proposed solution – make AI and Formal Logic test each other

References

Charles Antony Richard Hoare. “An axiomatic basis for computer programming”. In: Commun. ACM 12.10 (Oct. 1969), pp. 576–580. issn: 0001-0782. doi:10.1145/363235.363259. url: https://doi.org/10.1145/363235.363259 (visited on 04/12/2025).

Robert W. Floyd. “Assigning meanings to programs”. In: Program Verification: Fundamental Issues in Computer Science. Springer, 1967, pp. 65–81. url: https://people.eecs.berkeley.edu/~necula/Papers/FloydMeaning.pdf (visited on 07/23/2025).

Danny Dolev and Andrew Yao. “On the security of public key protocols”. In: IEEE Transactions on information theory 29.2 (1983), pp. 198–208. doi: 10.1109/TIT.1983.1056650.

Gavin Lowe. “Breaking and fixing the Needham-Schroeder Public-Key Protocol using FDR”. In: Tools and Algorithms for the Construction and Analysis of Systems. Ed. by Tiziana Margaria and Bernhard Steffen. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 147–166. isbn: 978-3-540-49874-2. url:https://doi.org/10.1007/3-540-61042-1_43 (visited on 03/09/2025).

Moses D. Liskov, Paul D. Rowe, and F. Javier Thayer. "Completeness of CPSA." MITRE Technical Report MTR110479, The MITRE Corporation, Bedford, Massachusetts: 2011. Retrieved on 10 NOV 25 from https://www.mitre.org/sites/default/files/pdf/12_0038.pdf

Szegedy, Christian, et al. “Going Deeper with Convolutions.” arXiv:1409.4842, arXiv, 17 Sept. 2014. *arXiv.org*, https://doi.org/10.48550/arXiv.1409.4842.

Tools

CaDiCaL Simplified Satisfiability Solver. C++. Version 3.0, 23 Dec. 2025, https://github.com/arminbiere/cadical

mitre/cpsa. 14 Feb. 2020, Scilab. The MITRE Corporation, 27 Feb. 2026. *GitHub*, https://github.com/mitre/cpsa

FDR4 — The CSP Refinement Checker. University of Oxford, https://cocotec.io/fdr/

Lean FRO. *Lean Programming Language and Theorem Prover*. 2025, https://lean-lang.org/

pysathq. p*ysathq/pysat*. 11 Feb. 2018, Python. 25 Feb. 2026. *GitHub*, https://github.com/pysathq/pysat

"Welcome to a World of Rocq." *Rocq*, 15 Sept. 2025, https://rocq-prover.org

Z3Prover/Z3. 26 Mar. 2015, C++. Z3 Theorem Prover, 1 Mar. 2026. *GitHub*, https://github.com/Z3Prover/z3